

Problem Set 7: Image Synthesis

Posted: Wednesday, Oct. 22, 2025

Due: Wednesday, Nov. 5, 2025

Submission instructions:

- Submit the following deliverables to [Canvas](#):
 - Colab notebook (`<username>_<umid>.ipynb`) containing all solutions and visualizations for problems 7.1 and 7.2. Ensure the filename matches the format.
 - PDF output (`<username>_<umid>.pdf`) of Colab notebook. For your convenience, we have included the PDF conversion script at the end of the notebook.

The starter code can be found at:

<https://drive.google.com/file/d/1j0a8Cloms0SpJGfunUYDvfa108VC3YYS/view?usp=sharing>

We recommend editing and running your code in Google Colab, although you are welcome to use your local machine instead.

Problem 7.1 Implement *pix2pix* (12 points)

In this problem set, we will implement an image-to-image translation program based on [pix2pix](#) [1]. We will train the pix2pix model on the `edges2shoes` dataset [1, 2] to translate shoe edge maps into full shoe images. The edges are automatically extracted from real shoe images.

Some example edge/image pairs are shown in Figure 1.

The pix2pix model is based on a conditional GAN (Figure 2). The generator G maps the source image x to a synthesized target image. The discriminator D takes both the source image and predicted target image as its inputs, and predicts whether the input is real or fake.



Figure 1: Example sketch-image pairs from the `edges2shoes` dataset. The edges are extracted with HED edge detector [3] from the raw shoe images. A model trained on this dataset can also work with user-provided sketches.

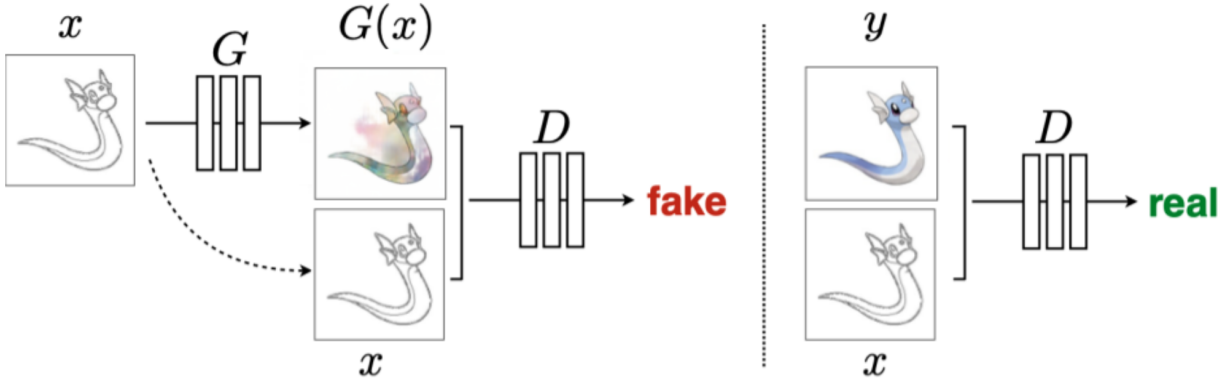


Figure 2: Conditional GAN for image translation. Training a conditional GAN to map edges→photo. The discriminator, D , learns to classify between fake (synthesized by the generator) and real edge, photo tuples. The generator, G , learns to fool the discriminator. Unlike an unconditional GAN, both the generator and discriminator observe the input edge map. The Pokémon images shown here come from the *Pokémon Images Dataset* [4, 5].

0. **(Optional, 0 points)** We will first build data loaders for training and testing. For the training process, we will use a batch size of 4. During testing, we will process 5 images in a single batch, so that we can visualize several results at once. Please complete the `Edges2Image` class and fill in the TODOs in that cell.

Hint: please use the `DataLoader` from `torch.utils.data`

1. **(6 points)** Next, we will define the network based on the architecture from the paper. The generator follows a U-net architecture, where the activations from the encoder are concatenated with the inputs to the decoder (Figure 3). We have included a toy U-net example in the Colab notebook.

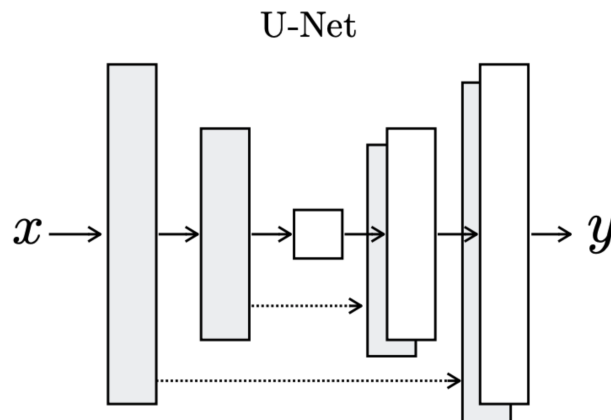


Figure 3: U-Net Architecture

As a reminder: let Ck denote a Convolution-BatchNorm-ReLU layer with k filters. All convolutions are 4×4 spatial filters applied with padding 1, and stride 2, except for the last 2 layers in the discriminator, which have stride 1. Convolutions in the encoder and the discriminator downsample the input by a factor of 2, while transposed convolutions in the decoder upsample the input by a factor of 2.

- (a) (**3 points**) Generator architectures The U-Net encoder-decoder architecture consists of:

U-Net encoder:

C64-C128-C256-C512-C512-C512-C512

U-Net decoder:

C512-C512-C512-C512-C256-C128-C64-C3

As a special case, batch normalization is not applied to the first and last layers in the encoder. All nonlinearities in the encoder are Leaky ReLUs, with slope 0.2, while the nonlinearities in the decoder are ReLUs. After the last layer in the decoder, a convolution is applied to map to the number of output channels, which is 3 in our problem, followed by a tanh function.

Please complete the generator class in the starter code.

Hint: you can use `torch.cat` to concatenate the decoder and the encoder inputs.

- (b) (**3 points**) Discriminator architectures

The discriminator architecture is:

C64-C128-C256-C512

As an exception to the above notation, batch normalization is not applied to the first and last layers. All nonlinearities are Leaky ReLUs, with slope 0.2. After the last layer, a convolution is applied to map to a 1-dimensional output, followed by a sigmoid function. Please complete the discriminator class in the starter code

2. (**1 point**) For optimization, we'll use the Adam optimizer. Adam is similar to SGD with momentum, but it also contains an adaptive learning rate for each model parameter. If you want to learn more about Adam, please refer to the deep learning book [6]. For our model training, we will use a learning rate of 0.0002, and momentum parameters $\beta_1 = 0.5$ and $\beta_2 = 0.999$. Please set up `G_optimizer` and `D_optimizer` in the train function.
3. (**4 points**) Now, we will implement the training routine and start training the models.

The conditional GAN (cGAN) loss function can be written as:

$$\mathcal{L}_{cGAN}(G, D) = \frac{1}{N} \sum_{i=1}^N \log D(x_i, y_i) + \frac{1}{N} \sum_{i=1}^N \log (1 - D(x_i, G(x_i))) \quad (1)$$

We also add an L1 loss to the total total loss function:

$$\mathcal{L}_{L1}(G) = \frac{1}{N} \sum_{i=1}^N \|y_i - G(x_i)\|_1 \quad (2)$$

Each iteration, we first train discriminator D by using the average loss of real images and fake images. We then train the generator G by using the following loss:

$$G^* = \arg \min_G \max_D \left(\mathcal{L}_{cGAN}(G, D) + \lambda \mathcal{L}_{L1}(G) \right) \quad (3)$$

You will train two different models: one with only L1 loss, the other with Equation 3 and $\lambda = 100$. Train the network for at least 20 epochs (10 epochs is enough for the model with only L1 loss). You are welcome to train longer, though, to potentially obtain better results.

Please complete the following tasks:

- In the specified text cell of the Colab notebook, comment on the difference between the generated images obtained from L1 only and L1 + cGAN.
- Show the history of the generator's BCE and L1 losses and the discriminator's loss vs. iteration of the $c = 100$ model in 3 separate plots.
- Show the history of the generator and the discriminator L1 loss vs. iteration of the L1 only model in 2 separate plots.
- In the specified text cell of the Colab notebook, comment on the difference among the history of loss plots for the L1 only and L1 + cGAN models. Specifically, what are the behaviors of BCE and L1 losses for the $c = 100$ model?

Note: training each epoch will take less than 2 minutes. If your training takes significantly longer than this, there is likely a mistake in your implementation.

4. (**Optional, 0 points**) After the pix2pix model has been trained on this dataset, we can apply the trained generative model to translate any user-provided sketch to a synthetic image. A shoe sketch drawn by a GSI and the synthesized shoe image are plotted in Figure 4.



Figure 4: Left: a sketch drawn by a GSI. Right: the synthesized shoe image.

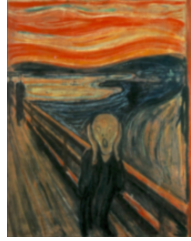
Please draw a shoe in the sketch panel we provide in the Colab notebook and translate it to a shoe image with the trained model. Feel free to post your generated image to a Piazza thread!

Problem 7.2 Style Transfer (Optional, 0 points)

In this problem, we will implement the loss functions for the neural artistic style transfer method of Gatys et al. [7]. We take two images as input: one that defines the *content* and another that defines the *style* (Figure 5). We have provided five paintings that define the styles, and images from COCO that define the content [8].



(a) Content image



(b) Style image



(c) Result image

Figure 5: **Style transfer.** We manipulate an image (a) to match the artistic style of (b), producing the resulting image (c).

Two losses are needed to accomplish style transfer: the content loss and the style loss. Both losses are calculated between the input image (the same as the content image or a random noise) and the reference image (style/content).

1. **(Optional, 0 points)** First, we will implement the content loss. This loss encourages the generated image to match the scene structure of the content image. We will implement this loss as the squared L2 distance between two convolutional feature maps. Given a feature map of the input image F^x and the feature map of the content image F^c , both of shape (C, H, W) , the content loss is calculated as follows:

$$\mathcal{L}_c = \sum_{c,i,j} (F_{c,i,j}^c - F_{c,i,j}^x)^2 \quad (4)$$

2. **(Optional, 0 points)** Next, we will implement the *style* loss. This loss encourages the texture of the resulting image to match the input style image. We compute a weighted, squared L2 distance between Gram matrices for several layers of the network.

The first step is to calculate the Gram matrix. **This is given to you.** Given a feature map F of size (C, H, W) , the Gram matrix $G \in R^{C \times C}$ computes the sum of products between channels. The entries k, l of the matrix are computed as:

$$G_{k,l} = \sum_{i,j} F_{k,i,j} F_{l,i,j} \quad (5)$$

The second step is to compare the generated image's Gram matrix with that of the input style image. Define the Gram matrix of the input image feature map and style image feature map at the l^{th} layer as $G^{x,l}$ and $G^{s,l}$. Style loss at the l^{th} layer is:

$$\mathcal{L}_s^l = w^l \sum_{i,j} (G_{i,j}^{x,l} - G_{i,j}^{s,l})^2 \quad (6)$$

where w^l is the weight of layer l .

The total style loss is a sum over all style layers:

$$\mathcal{L}_s = \sum_l \mathcal{L}_s^l \quad (7)$$

Please write the code for computing the style loss, given the Gram matrix code

3. **(Optional, 0 points)** Finally, we will try varying the layer used to compute the content loss. Try several different layers $l = 0, 1, 2, \dots, 12$ and report which one gave you the best qualitative results.
4. **(Optional, 0 points)** Upload any of your own images or try out other images in the datasets and perform style transfer. We encourage you to post your art to the Piazza thread!

Acknowledgements. Part of the homework dataset and the starter code are taken from a previous EECS 498 class taught by Justin Johnson, which itself was adapted from CS231n at Stanford University by Fei-Fei Li, Justin Johnson and Serena Yeung. Please feel free to similarly re-use our problems while crediting these other classes.

References

- [1] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A. Efros. Image-to-image translation with conditional adversarial networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [2] A. Yu and K. Grauman. Fine-grained visual comparisons with local learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2014.
- [3] Saining Xie and Zhuowen Tu. Holistically-nested edge detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015.
- [4] Pokemon images dataset. <https://www.kaggle.com/kvpratama/pokemon-images-dataset>.
- [5] zaidalyafeai.github.io. <https://github.com/zaidalyafeai/zaidalyafeai.github.io/tree/master/pix2pix/datasets>, 2018.
- [6] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*, volume 1. MIT Press, Cambridge, 2016.
- [7] Leon A. Gatys, Alexander S. Ecker, and Matthias Bethge. A neural algorithm of artistic style. *arXiv preprint arXiv:1508.06576*, 2015.
- [8] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740–755. Springer, 2014.